

Проектирование микропроцессорной системы на базе МК i8051

Описание программной части

Листов 26

Инд. № подл.	Подп. и дата
Взам. инд. №	Инд. № дубл.
Подп. и дата	Подп. и дата

Выполнил: Грошев А.В.
Группа: А-06-23
Вариант: 1

Проверил: Ключанский А.А.

Москва, 2026

АННОТАЦИЯ

В данном документе приведено описание программы «Программа управления объектом», предназначенной для управления объектом микропроцессорной системой (МПС) на базе микроконтроллера AT89S53: ввода данных с клавиатуры и от внешнего устройства, формирования управляющих сигналов Y_1 и Y_2 и вывода информации на семисегментный индикатор.

Для функционирования данной программы необходима МПС, собранная по схеме электрической принципиальной варианта 1 (пояснительная записка по аппаратной части).

Исходным языком программы «Программа управления объектом» является ассемблер микроконтроллеров семейства MCS-51.

Основными функциями программы являются: инициализация МПС, сканирование клавиатуры, работа с кольцевым буфером на 1210 байт, выработка stroba Y_1 длительностью 176 мс, выработка сигнала Y_2 со стробом 1536 мкс, индикация на одном разряде.

Оформление программного документа «Описание программы» произведено по требованиям ЕСПД (ГОСТ 19.101-77, ГОСТ 19.103-77, ГОСТ 19.104-78, ГОСТ 19.105-78, ГОСТ 19.106-78, ГОСТ 19.402-78, ГОСТ 19.604-78).

СОДЕРЖАНИЕ

1 Общие сведения.....	4
2 Функциональное назначение.....	5
2.1 Классы решаемых задач.....	5
2.2 Назначение программы.....	5
2.3 Сведения о функциональных ограничениях на применение.....	5
3 Описание логической структуры.....	6
3.1 Алгоритм программы.....	6
3.2 Структура программы с описанием функций составных частей и связи между ними	18
3.3 Связи программы с другими программами.....	19
4 Входные данные.....	20
4.1 Клавиатура.....	20
4.2 Сигнал X2.....	20
5 Выходные данные.....	21
5.1 Характер и организация выходных данных.....	21
5.2 Формат, описание и способ кодирования выходных данных.....	21
ПЕРЕЧЕНЬ ПРИНЯТЫХ СОКРАЩЕНИЙ.....	23
ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ.....	24
ПРИЛОЖЕНИЕ А. Листинги программ.....	25

1 Общие сведения

Обозначение и наименование программы — «Программа управления объектом», вариант 1, группа А-06-23.

Программа низкого уровня не требует для своей работы иных программ. Исходным языком программирования является ассемблер MCS-51 (синтаксис Intel ASM51, совместимый с Keil A51 и MCU 8051 IDE).

Среда разработки и инструментальные средства: интегрированная среда Keil μ Vision V5 (Keil Software, пакет PK51); ассемблер — A51; компоновщик — BL51.

2 Функциональное назначение

2.1 Классы решаемых задач

- инициализация МПС;
- сканирование клавиатуры — ввод сигнала X_1 (число, введённое с клавиатуры);
- формирование сигнала Y_1 (строб заданной длительности);
- ввод сигнала X_2 (сохранение данных в кольцевом буфере);
- формирование сигнала Y_2 и сопровождающего строба;
- вывод символа на индикатор.

2.2 Назначение программы

МПС принимает информацию $\{X\}$ об объекте управления от датчиков, вырабатывает управляющие воздействия $\{Y\}$ в соответствии с заданным алгоритмом управления и подаёт их на исполнительные механизмы. Программа разбита на подзадачи по функциональному признаку:

- инициализация МПС;
- обработка прерываний: опрос клавиатуры, запись в буфер, таймеры генераторов сигналов Y ;
- определение кода нажатой клавиши;
- вывод данных на семисегментный индикатор;
- формирование сигнала Y_2 ;
- обеспечение работы кольцевого буфера (запись и чтение отсчётов X_2).

В соответствии с вариантом (табл. 1 ТЗ) разработан код трёх программ:

- программа 1 — обработчик прерывания клавиатуры (сканирование матрицы), код нажатой клавиши в аккумуляторе или 0FFh;
- программа 2 — запись в кольцевой буфер (данные через аккумулятор) с управлением флагами пустоты и переполнения;
- программа 3 — выработка строба Y_1 длительностью T_1 (прерывание таймера).

2.3 Сведения о функциональных ограничениях на применение

В качестве основы для построения МПС выбран микроконтроллер семейства MCS-51 — AT89S53, тактовая частота 12 МГц (машинный цикл 1 мкс). Таймер 2 и сторожевой таймер не используются (совместимость с i8051).

Интерфейсы:

- входные данные X_2 — 8-разрядное целое без знака по независимой параллельной шине со стробом записи низкого уровня в двухпортовое ОЗУ, нерегулярно, до 4 отсчётов в секунду;
- входные данные X_1 — десятичная цифра 0–9 с матричной клавиатуры 3×4 (сканирование по столбцам);
- выходной сигнал Y_2 — 8-разрядный параллельный код в выходном регистре со стробом низкого уровня длительностью 1536 мкс на выводе P1.2;
- выходной сигнал Y_1 — строб низкого уровня длительностью 176 мс на выводе P1.1;
- индикация — один семисегментный индикатор (общий катод), время индикации символа не более $T_3 = 2,150$ с.

3 Описание логической структуры

3.1 Алгоритм программы

✍️ ДОПИШИ (p2.alg в students/.../pz/): опиши основной цикл: что делается после сброса (инициализация), что крутится в основном цикле и какие действия выполняются в прерываниях (клавиатура — INT0, X2 — INT1, таймеры). 1–2 абзаца, со ссылками на рисунки ниже.

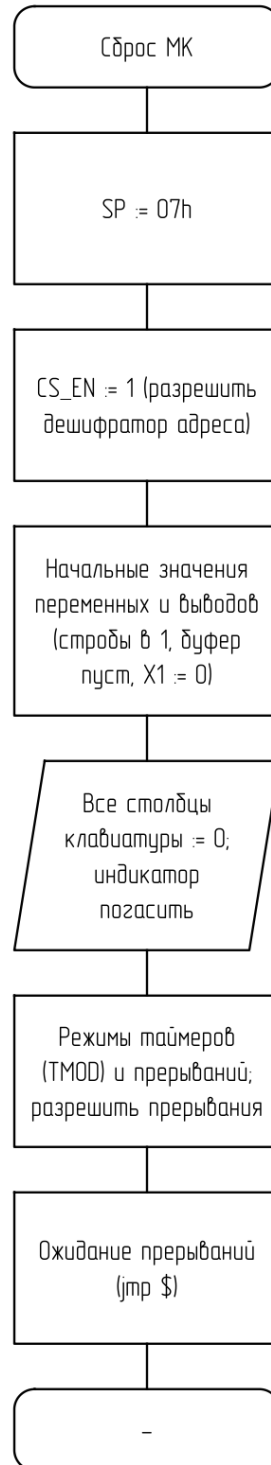


Рисунок 1 — Схема алгоритма: Инициализация МПС и основная программа

Схема «Инициализация МПС и основная программа» выполняется так: вход — «Сброс МК»; SP := 07h; CS_EN := 1 (разрешить дешифратор адреса); начальные значения переменных и выводов (стробы в 1, буфер пуст, X1 := 0); обмен с устройством: все столбцы клавиатуры := 0;

индикатор погасить; режимы таймеров (TMOD) и прерываний; разрешить прерывания; ожидание прерываний (jmp \$).

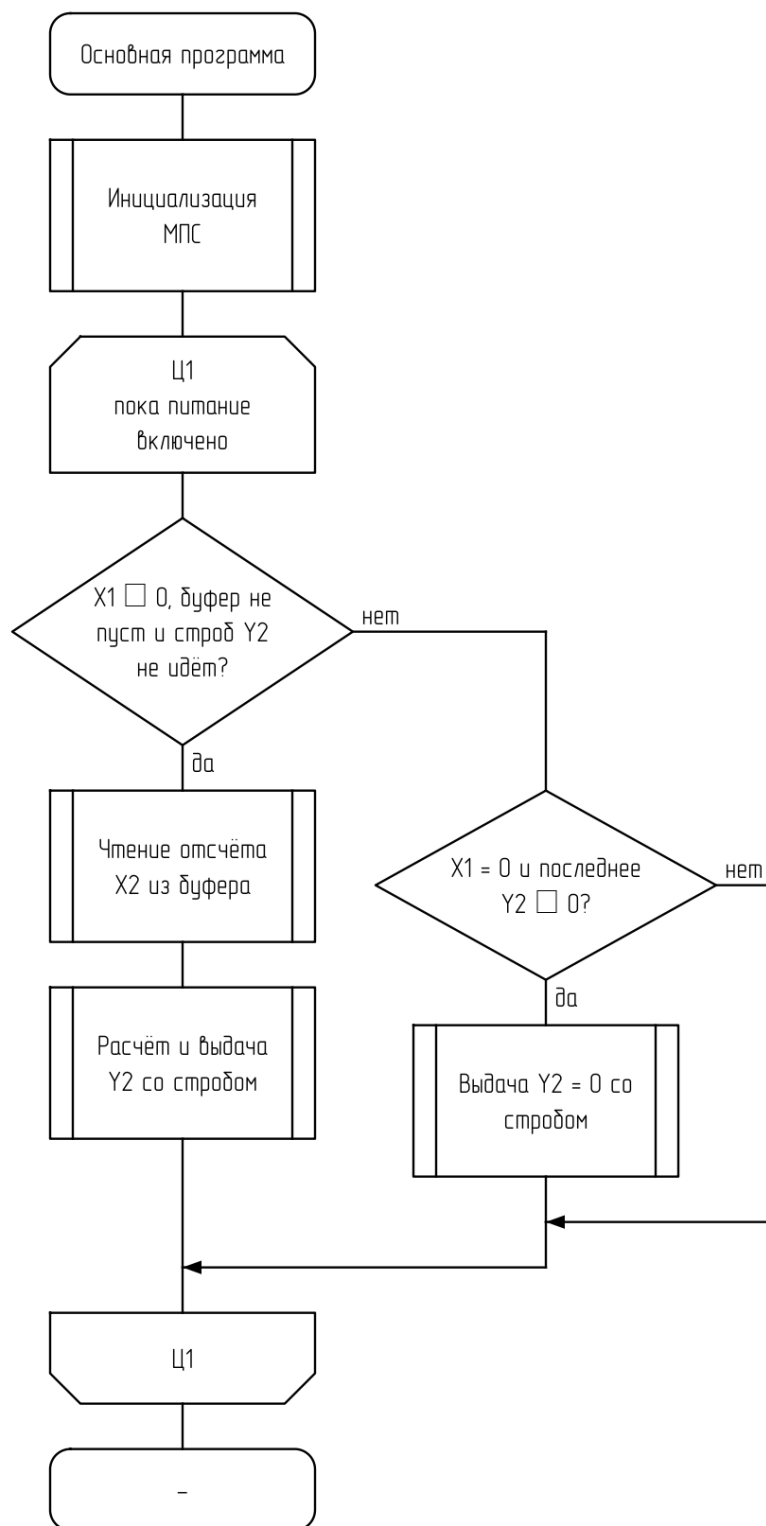


Рисунок 2 — Схема алгоритма: Основной цикл программы

Схема «Основной цикл программы» выполняется так: вход — «Основная программа»; вызывается подпрограмма: инициализация МПС; в цикле Ц1 (пока питание включено) повторяется: проверяется условие « $X1 \neq 0$, буфер не пуст и строб Y2 не идёт»: если да — вызывается подпрограмма: чтение отсчёта X2 из буфера, вызывается подпрограмма: расчёт и выдача Y2 со стробом, иначе — проверяется условие « $X1 = 0$ и последнее Y2 $\neq 0$ »: если да — вызывается подпрограмма: выдача Y2 = 0 со стробом.

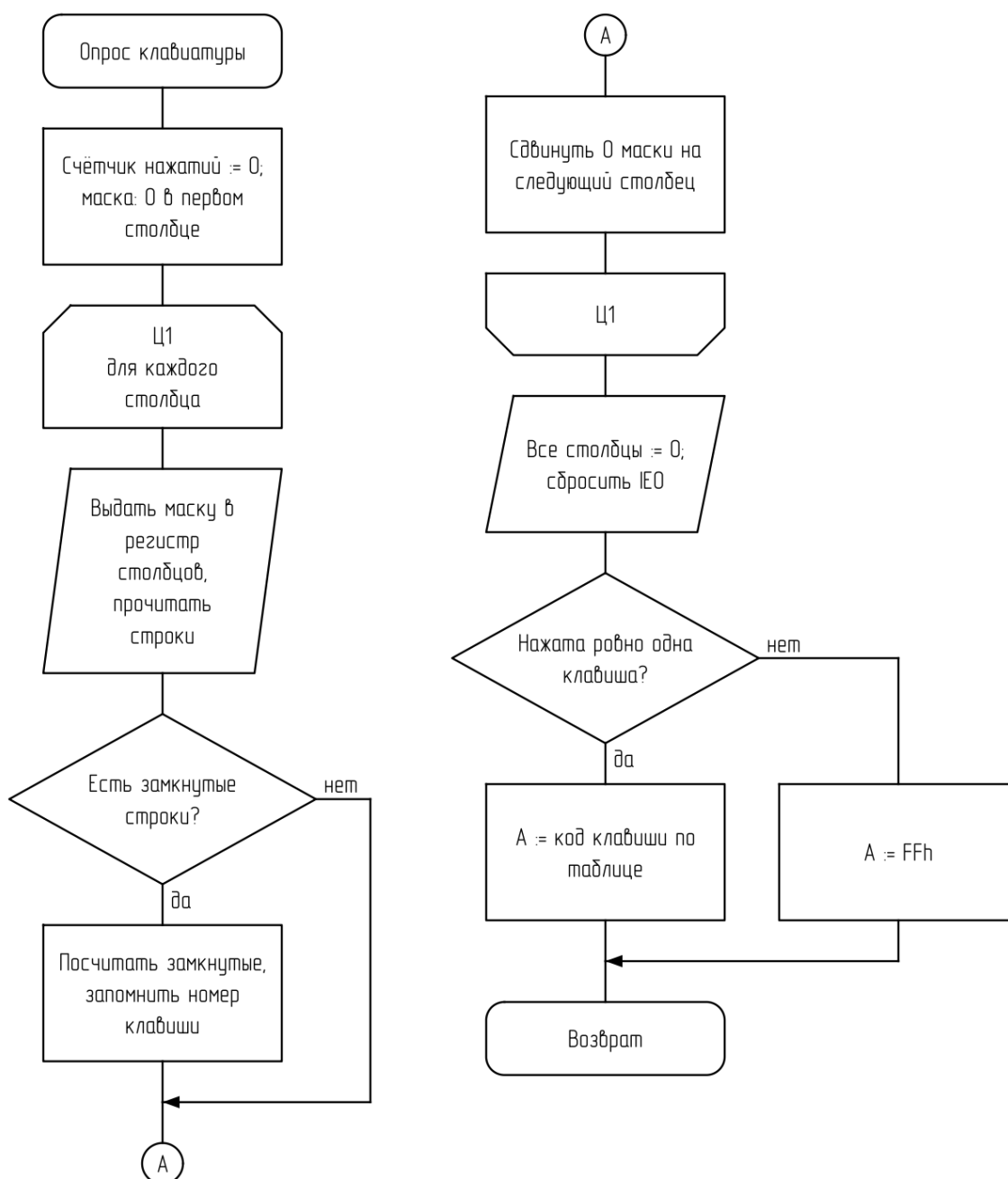


Рисунок 3 — Схема алгоритма: Опрос клавиатуры (обработчик INT0)

Схема «Опрос клавиатуры (обработчик INT0)» выполняется так: вход — «Опрос клавиатуры»; счётчик нажатий := 0; маска: 0 в первом столбце; в цикле Ц1 (для каждого столбца) повторяется: обмен с устройством: выдать маску в регистр столбцов, прочитать строки, проверяется условие «Есть замкнутые строки»: если да — посчитать замкнутые, запомнить номер клавиши, сдвинуть 0 маски на следующий столбец; обмен с устройством: все столбцы := 0; сбросить IE0; проверяется условие «Нажата ровно одна клавиша»: если да — A := код клавиши по таблице, иначе — A := FFh; в конце — возврат.

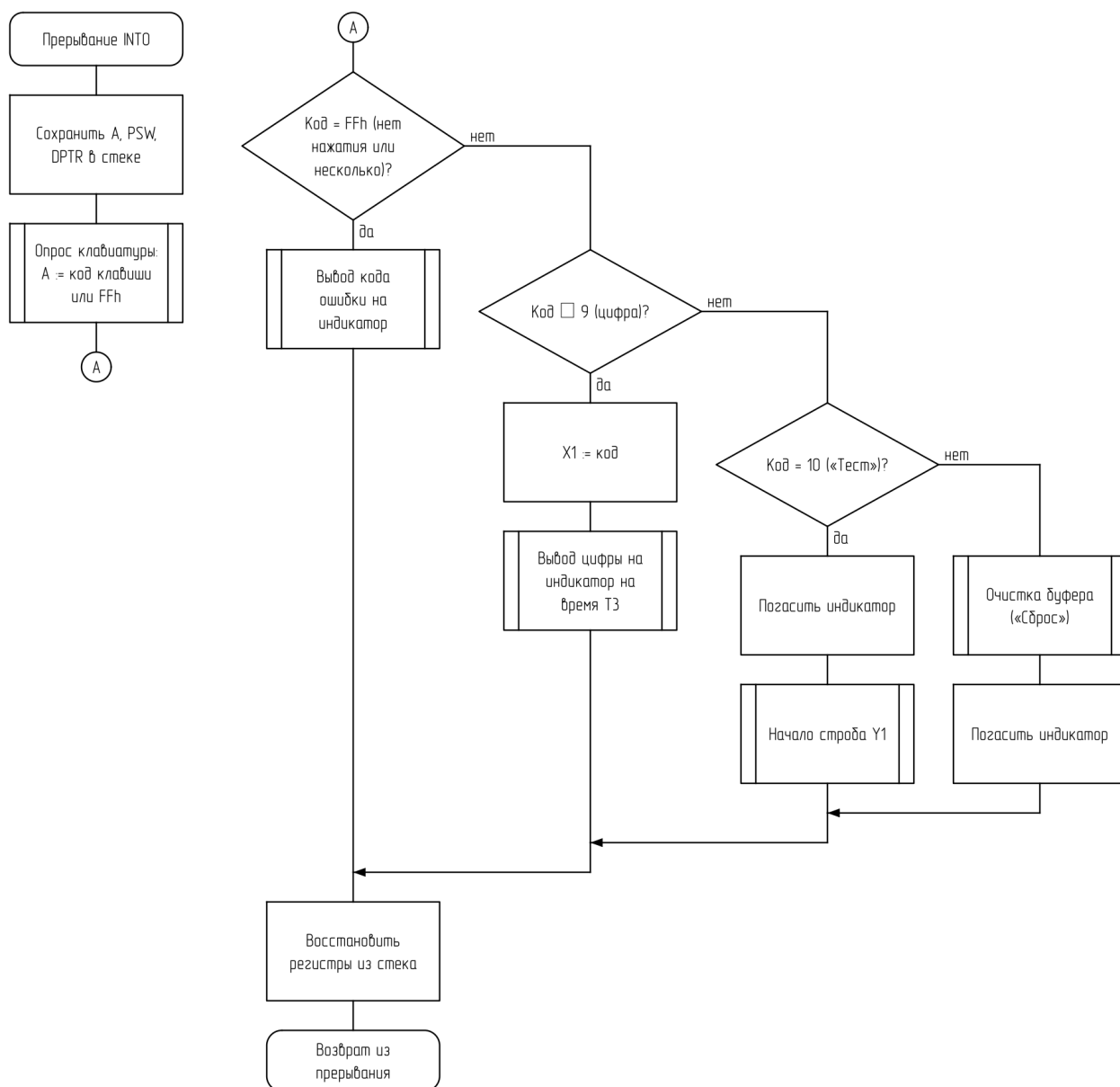


Рисунок 4 — Схема алгоритма: Обработчик прерывания INT0: реакция на клавишу

Схема «Обработчик прерывания INT0: реакция на клавишу» выполняется так: вход — «Прерывание INT0»; сохранить A, PSW, DPTR в стеке; вызывается подпрограмма: опрос клавиатуры: A := код клавиши или FFh; проверяется условие «Код = FFh (нет нажатия или несколько)»: если да — вызывается подпрограмма: вывод кода ошибки на индикатор, иначе — проверяется условие «Код ≤ 9 (цифра)»: если да — X1 := код, вызывается подпрограмма: вывод цифры на индикатор на время T3, иначе — проверяется условие «Код = 10 («Тест»)»: если да — погасить индикатор, вызывается подпрограмма: начало строба Y1, иначе — вызывается подпрограмма: очистка буфера («Сброс»), погасить индикатор; восстановить регистры из стека; в конце — возврат из прерывания.



Рисунок 5 — Схема алгоритма: Запись отсчёта в кольцевой буфер

Схема «Запись отсчёта в кольцевой буфер» выполняется так: вход — «Запись в буфер»; проверяется условие «Буфер полон (голова = хвост и не пуст)»: если да — хвост := следующая ячейка (самый старый отсчёт теряется), флаг переполнения := 1; обмен с устройством: `товх @DPTR` по адресу «головы»: правый порт записывает байт X2 с разъёма; голова := следующая ячейка (после последней — первая); флаг пустоты := 0; в конце — возврат.

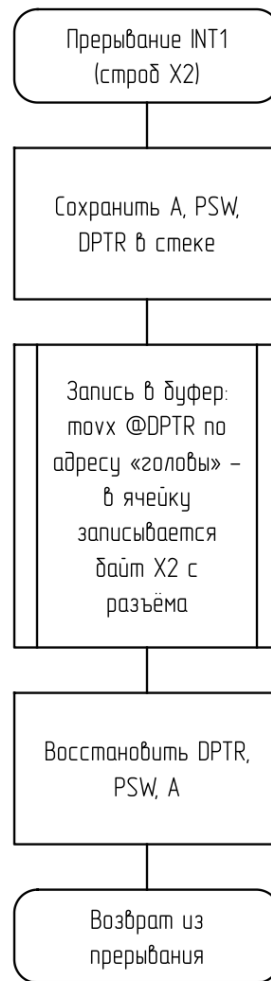


Рисунок 6 — Схема алгоритма: Обработчик прерывания INT1: приём отсчёта X2 от внешнего устройства

Схема «Обработчик прерывания INT1: приём отсчёта X2 от внешнего устройства» выполняется так: вход — «Прерывание INT1 (строб X2)»; сохранить A, PSW, DPTR в стеке; вызывается подпрограмма: запись в буфер: `movx @DPTR` по адресу «головы» — в ячейку записывается байт X2 с разъёма; восстановить DPTR, PSW, A; в конце — возврат из прерывания.

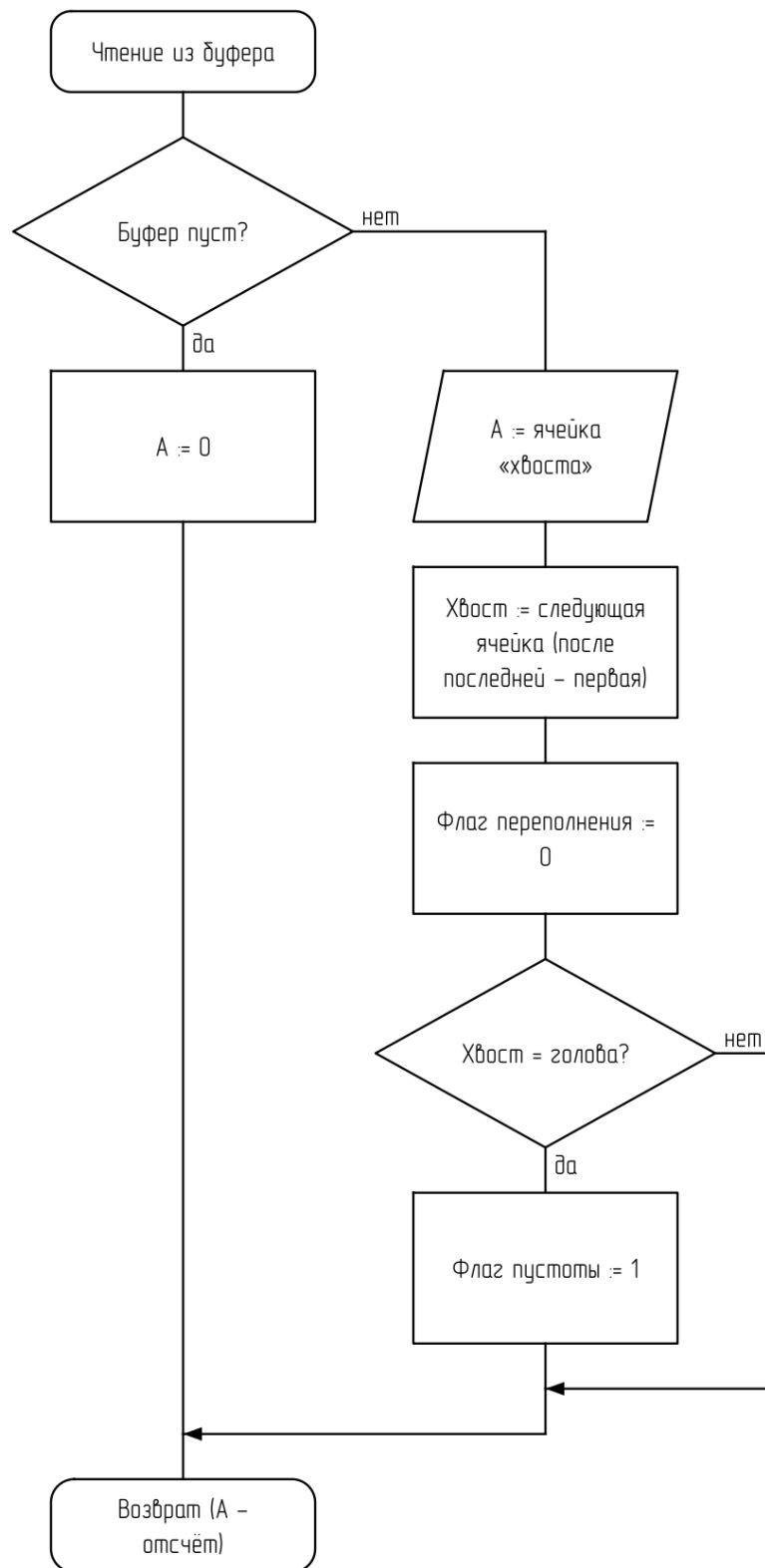


Рисунок 7 — Схема алгоритма: Чтение отсчёта из кольцевого буфера

Схема «Чтение отсчёта из кольцевого буфера» выполняется так: вход — «Чтение из буфера»; проверяется условие «Буфер пуст»: если да — $A := 0$, иначе — обмен с устройством: $A := \text{ячейка «хвоста»}$, $\text{хвост} := \text{следующая ячейка (после последней — первая)}$, $\text{флаг переполнения} := 0$, проверяется условие «Хвост = голова»: если да — $\text{флаг пустоты} := 1$; в конце — возврат (A — отсчёт).

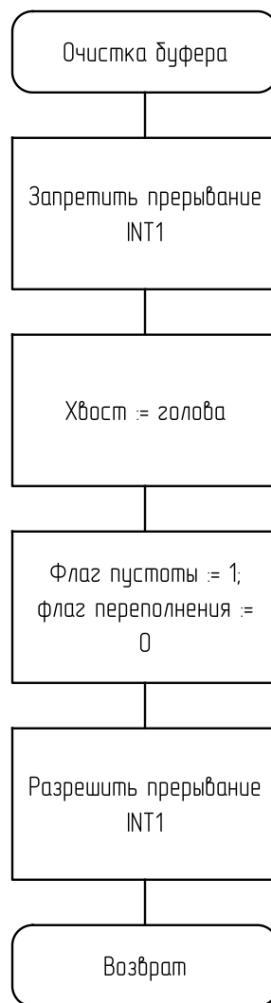


Рисунок 8 — Схема алгоритма: Очистка кольцевого буфера (клавиша «Сброс»)

Схема «Очистка кольцевого буфера (клавиша «Сброс»)» выполняется так: вход — «Очистка буфера»; запретить прерывание INT1; хвост := голова; флаг пустоты := 1; флаг переполнения := 0; разрешить прерывание INT1; в конце — возврат.

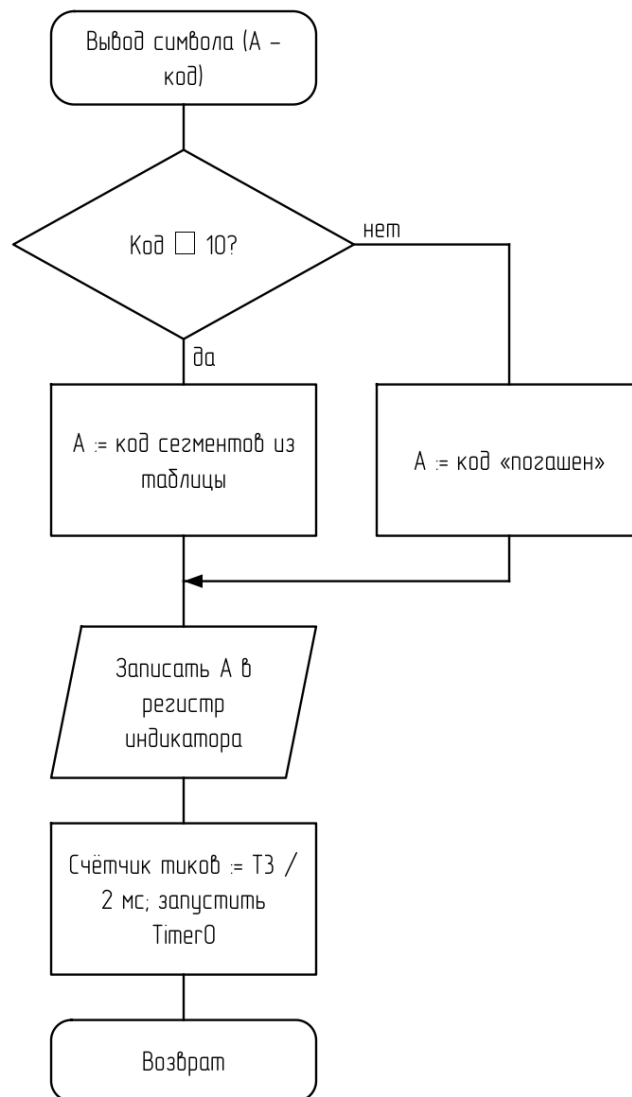


Рисунок 9 — Схема алгоритма: Вывод символа на индикатор

Схема «Вывод символа на индикатор» выполняется так: вход — «Вывод символа (A — код)»; проверяется условие «Код ≤ 10 »: если да — $A :=$ код сегментов из таблицы, иначе — $A :=$ код «погашен»; обмен с устройством: записать A в регистр индикатора; счётчик тиков $:= T3 / 2$ мс; запустить Timer0; в конце — возврат.

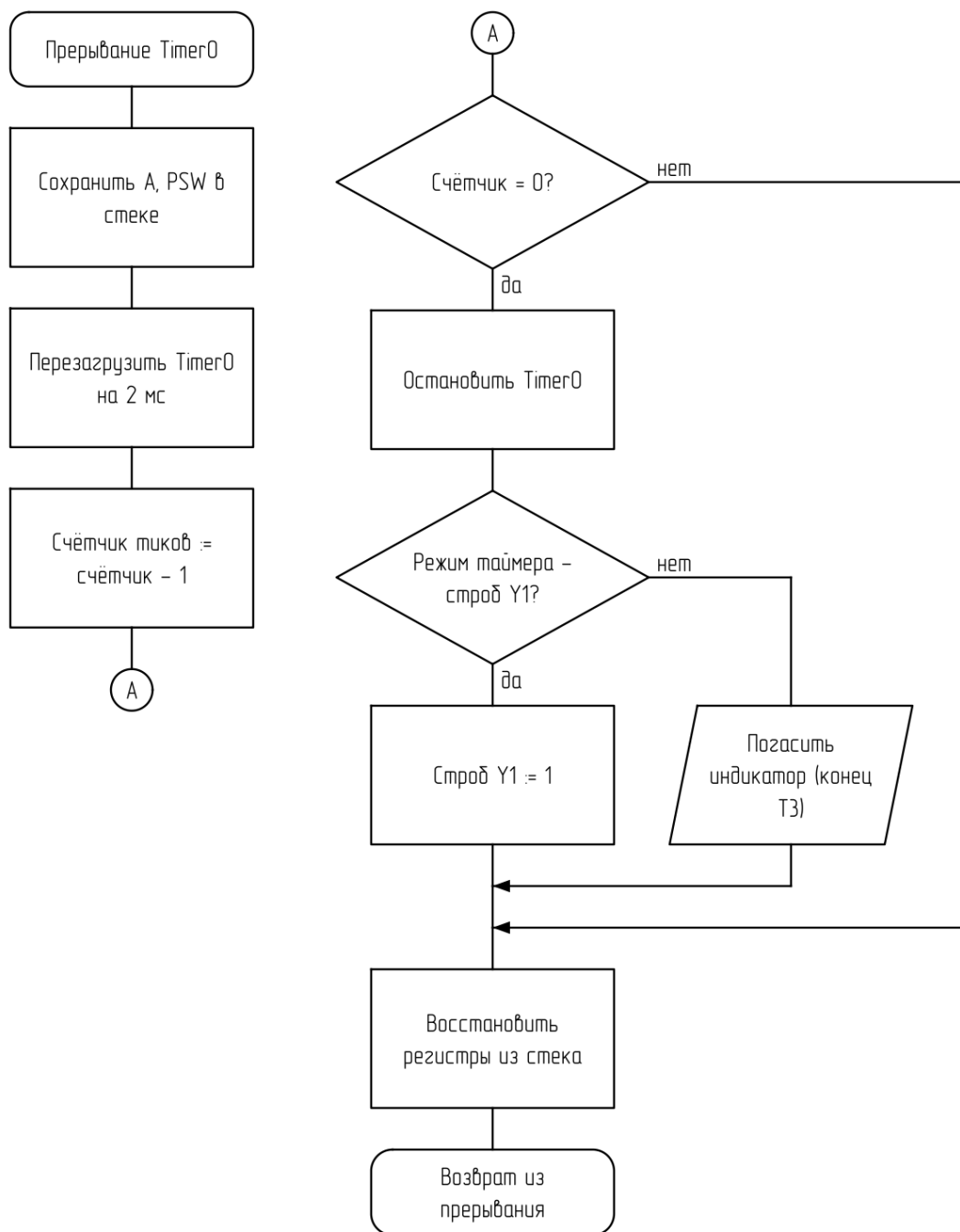


Рисунок 10 — Схема алгоритма: Обработчик прерывания Timer0: отсчёт T1 и T3

Схема «Обработчик прерывания Timer0: отсчёт T1 и T3» выполняется так: вход — «Прерывание Timer0»; сохранить A, PSW в стеке; перезагрузить Timer0 на 2 мс; счётчик тиков := счётчик - 1; проверяется условие «Счётчик = 0»: если да — остановить Timer0, проверяется условие «Режим таймера — строб Y1»: если да — строб Y1 := 1, иначе — обмен с устройством: погасить индикатор (конец T3); восстановить регистры из стека; в конце — возврат из прерывания.

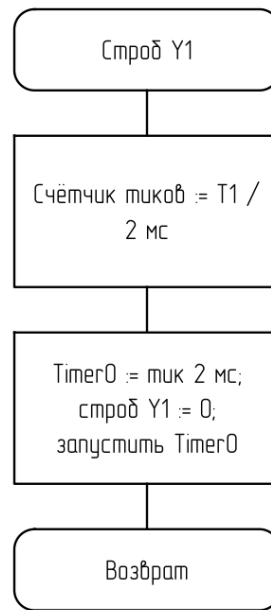


Рисунок 11 — Схема алгоритма: Начало строба Y1

Схема «Начало строба Y1» выполняется так: вход — «Строб Y1»; счётчик тиков := $T1 / 2$ мс; Timer0 := тик 2 мс; строб Y1 := 0; запустить Timer0; в конце — возврат.

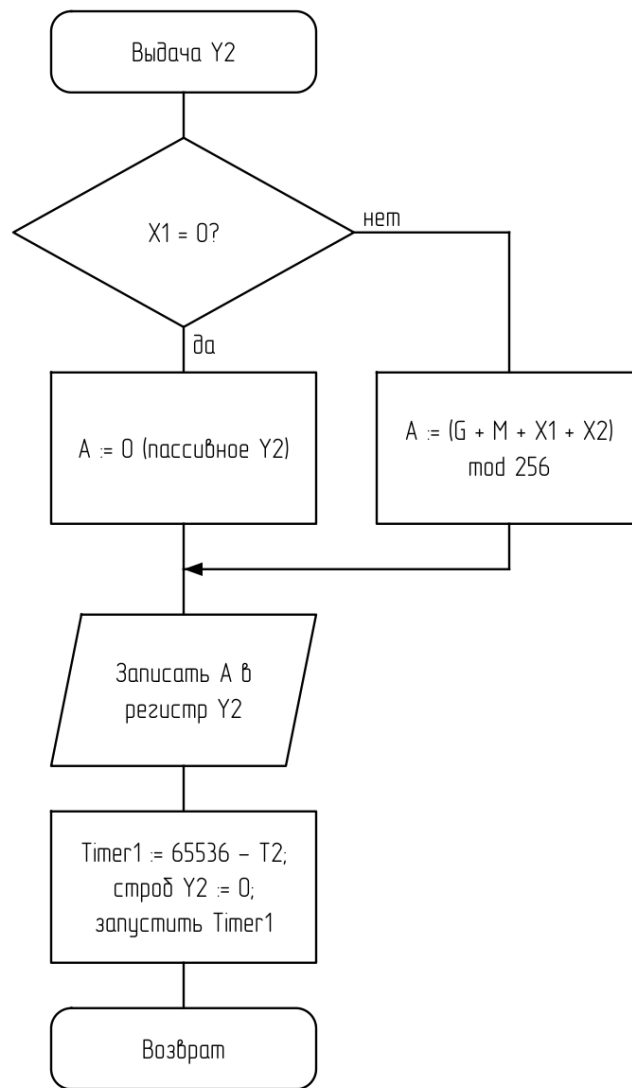


Рисунок 12 — Схема алгоритма: Расчёт и выдача сигнала Y2

Схема «Расчёт и выдача сигнала Y2» выполняется так: вход — «Выдача Y2»; проверяется условие «X1 = 0»: если да — $A := 0$ (пассивное Y2), иначе — $A := (G + M + X1 + X2) \bmod 256$; обмен с устройством: записать A в регистр Y2; $Timer1 := 65536 - T2$; строб Y2 := 0; запустить Timer1; в конце — возврат.

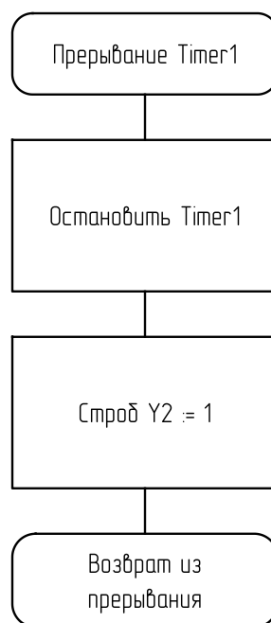


Рисунок 13 — Схема алгоритма: Обработчик прерывания Timer1: конец строга Y2

Схема «Обработчик прерывания Timer1: конец строга Y2» выполняется так: вход — «Прерывание Timer1»; остановить Timer1; строб Y2 := 1; в конце — возврат из прерывания.

3.2 Структура программы с описанием функций составных частей и связи между ними

Программа состоит из инициализации, основной программы (ожидание прерываний) и подпрограмм, описанных ниже. Описание каждой подпрограммы взято из заголовочного комментария в исходном тексте (приложение А).

3.2.1 Подпрограмма KbScan (программа 1)

Назначение: сканирование матрицы клавиатуры по столбцам (обработчик INT0).

Входные данные: нет.

Выходные данные: A — код единственной нажатой клавиши по табл. 3 T3 или 0FFh. TODO: портит

3.2.2 Подпрограмма BufWrite (программа 2)

Назначение: запись отсчёта в кольцевой буфер; если буфер полон — затирается самый старый.

Входные данные: A — отсчёт.

Выходные данные: F_EMPTY = 0; F_OVF = 1, если отсчёт затёр самый старый. TODO: что портит.

3.2.3 Подпрограмма Y1Start (программа 3)

Назначение: начало строга Y1 = 0 длительностью T1 на PIN_Y1; конец — в обработчике Timer0. Вход, выход: нет. TODO: что портит.

3.2.4 Обработчик прерывания Timer0 — T0Isr (программа 3)

Назначение: тик Timer0: перезагрузка, счёт тиков, по окончании T1 — PIN_Y1 = 1.

Распределение ресурсов микроконтроллера — прерывания, адреса внешних устройств и переменные — приведено в таблицах 1–3.

Таблица 1 — Прерывания

Источник	Вектор	Назначение	Запуск
INT0 (P3.2)	0003h	нажатие клавиши	по спаду

Источник	Вектор	Назначение	Запуск
Timer0	000Bh	отсчёт T_1 (176 мс) и T_3 (2150 мс), тик 2 мс	переполнение
INT1 (P3.3)	0013h	строб записи X_2	по спаду
Timer1	001Bh	отсчёт T_2 (1536 мкс)	переполнение
UART	0023h	не используется (заглушка)	—

Таблица 2 — Адреса внешних устройств (обращение командами movx при P3.4 = 1)

Устройство	Выход дешифратора	Адреса
Индикатор	Y2	4000h–5FFFh
Клавиатура	Y4	8000h–9FFFh
Буфер (IDT7005)	Y1	2000h–3FFFh
Регистр Y2	Y3	6000h–7FFFh

Таблица 3 — Переменные во внутреннем ОЗУ

Имя	Адрес	Назначение
HEAD_L, HEAD_H	38h, 39h	«голова» — адрес ячейки буфера для следующей записи
TAIL_L, TAIL_H	3Ah, 3Bh	«хвост» — адрес ячейки буфера для следующего чтения
F_EMPTY	20h.1 (бит 01h)	флаг пустоты буфера
F_OVF	20h.7 (бит 07h)	флаг переполнения буфера

3.3 Связи программы с другими программами

Программа «Программа управления объектом» в ходе своей работы не использует других программ.

4 Входные данные

4.1 Клавиатура

Клавиатура — матрица 3×4 (3 столбцов × 4 строк), сканирование по столбцам: в регистр столбцов (адрес 8000h, разряды D0...D2) выдаётся 0 в опрашиваемый столбец, строки читаются через повторитель по тому же адресу (разряды D0...D3, 0 — контакт замкнут). Нажатие любой клавиши при всех столбцах в 0 вызывает прерывание INT0. Коды клавиш (табл. 3 ТЗ) и их расположение — в таблице 4.

Таблица 4 — Расположение клавиш и их коды («клавиша» — код)

	Col1	Col2	Col3
Row1	«1» — 1	«2» — 2	«3» — 3
Row2	«4» — 4	«5» — 5	«6» — 6
Row3	«7» — 7	«8» — 8	«9» — 9
Row4	«0» — 0	«Т» — 10	«С» — 11

Результат опроса — код единственной нажатой клавиши (0–9 — цифры, 10 — «Тест», 11 — «Сброс») или 0FFh, если не нажато ничего или нажато больше одной клавиши. Цифра становится значением X_1 (начальное значение $X_1 = 0$).

4.2 Сигнал X2

X_2 — 8-разрядное целое без знака. Внешнее устройство записывает отсчёт в двухпортовое ОЗУ IDT7005 через свой порт и сопровождает запись стробом, который вызывает прерывание INT1. Отсчёты хранятся в кольцевом буфере на 1210 байт (адреса 2000h–24B9h).

5 Выходные данные

5.1 Характер и организация выходных данных

Индикация. На индикаторе отображается последняя введённая цифра — до ввода следующей, но не дольше $T_3 = 2,150$ с; при ошибке — код ошибки с горящей точкой др. Начальное состояние — индикатор погашен.

Кольцевой буфер во внешнем ОЗУ. Ячейки 2000h–24B9h. «Голова» — адрес ячейки для следующей записи, «хвост» — для следующего чтения; после последней ячейки следует первая. Буфер пуст — флаг пустоты = 1 (голова = хвост). Если буфер заполнен, новый отсчёт затирает самый старый (хвост сдвигается) и устанавливается флаг переполнения; после чтения флаг переполнения сбрасывается.

Управляющие сигналы. Y_1 — строб низкого уровня длительностью 176 мс по клавише «Тест». $Y_2 = (6 + 1 + X_1 + X_2) \bmod 256$ при $X_1 \neq 0$, иначе $Y_2 = 0$ (пассивное состояние); вырабатывается, пока в буфере есть данные и $X_1 \neq 0$. Каждая запись Y_2 в выходной регистр сопровождается стробом длительностью 1536 мкс; до записи регистр выдаёт предыдущее значение.

Расчётные параметры работы программы. Константы таймеров (кварц 12 МГц, машинный цикл $T_{\text{мц}} = 12 / f = 1$ мкс, режим 1 — 16 бит):

ТН1:TL1 = 65536 – $T_2 / T_{\text{мц}} = 65536 - 1536 = 64000 = 0FA00h$ (строб Y_2 , таймер 1)

ТН0:TL0 = 65536 – $2 \text{ мс} / T_{\text{мц}} = 65536 - 2000 = 63536 = 0F830h$ (тик 2 мс, таймер 0)

$N_{T1} = T_1 / 2 \text{ мс} = 88$; $N_{T3} = T_3 / 2 \text{ мс} = 1075$ (число тиков)

Минимальное время заполнения буфера (при $X_1 = 0$ сигнал Y_2 пассивен и буфер не читается, отсчёты поступают с максимальной интенсивностью):

$t_{\text{min}} = V / f_{X2} = 1210 / 4 = 302,5$ с (5,0 мин).

Максимальная глубина стека для программы 1 — 0 байт (SP от 07h до 07h). Значение получено прогоном программы в симуляторе на всех сценариях проверки (каждая клавиша, одновременные нажатия, помеха на INT0) и складывается из адреса возврата при входе в обработчик INT0 (2 байта), адреса возврата при вызове процедуры опроса (2 байта) и сохраняемых в стеке регистров.

5.2 Формат, описание и способ кодирования выходных данных

Код символа (табл. 2 ТЗ) преобразуется в код сегментов по таблице 5: разряды D0...D7 регистра индикатора соответствуют сегментам a, b, c, d, e, f, g, др; индикатор с общим катодом — сегмент горит при 1.

Таблица 5 — Коды сегментов индикатора (др g f e d c b a)

Код	Символ	Двоичный	Шестн.
0	«0»	00111111	3Fh
1	«1»	00000110	06h
2	«2»	01011011	5Bh
3	«3»	01001111	4Fh
4	«4»	01100110	66h
5	«5»	01101101	6Dh
6	«6»	01111101	7Dh
7	«7»	00000111	07h
8	«8»	01111111	7Fh
9	«9»	01101111	6Fh
10	«Е»	01111001	79h
11 и больше	погашен	00000000	00h

Таблица 6 — Коды ошибок (символ с горящей точкой dr, индикация до нажатия любой клавиши, но не дольше T3)

Индикация	Смысл	Когда выставляется
«1.»	нажато несколько клавиш	опрос клавиатуры нашёл больше одного замкнутого контакта
«2.»	нажатие не распознано	прерывание пришло, а замкнутых контактов нет
«3.»	переполнение буфера	запись X_2 затёрла непрочитанный отсчёт
«4.»	повторный «Тест»	«Тест» нажат, пока идёт строб Y_1

 **ДОПИШИ** (p2.errors в students/.../pz/): проверь таблицу кодов ошибок выше: оставь те, что обрабатывает твоя программа, и одним-двумя предложениями опиши реакцию на ошибку (или напиши «таблица соответствует программе»).

Сигнал Y_2 выдаётся 8-разрядным кодом в выходной регистр (адрес 6000h), строб — на выводе P1.2; сигнал Y_1 — на выводе P1.1.

В 16-разрядном режиме (режим 1) таймер 0 перезагружается в обработчике прерывания; задержка входа в прерывание (несколько машинных циклов на тик) компенсируется поправкой к загружаемому значению или учитывается как погрешность длительности. В режиме 2 (8 разрядов с автоперезагрузкой) перезагрузку выполняет сам таймер, а тик набирается программным счётчиком переполнений.

ПЕРЕЧЕНЬ ПРИНЯТЫХ СОКРАЩЕНИЙ

Таблица 7 — Сокращения

Сокращение	Расшифровка
МК	микроконтроллер
МПС	микропроцессорная система
ОЗУ	оперативное запоминающее устройство
ПЗУ	постоянное запоминающее устройство
ЕСПД	единая система программной документации
ТЗ	техническое задание
CS	Chip Select, выбор микросхемы
IRQ	запрос прерывания
SP	указатель стека
DPTR	16-разрядный регистр-указатель данных
dp	десятичная точка индикатора

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

[illegible]

ПРИЛОЖЕНИЕ А

Листинги программ

Программа 1 — файл «Грошев АВ-код-1»

```
; Грошев А.В., А-06-23, 1, v1
$INCLUDE (vars.inc) ; адреса, биты, константы варианта; в файл для робота mpcode вклеит его текст
DSEG AT 30h
; здесь объявление переменных (если нужны)
CSEG AT 0h
    sjmp START
org 03h ; INT0 — нажатие клавиши: процедура вызывается отсюда
    call KbScan ; %proc%
    reti
org 0bh ; "заглушка" для Tmr0_ovf
    nop
    reti
org 13h ; "заглушка" для INT1
    nop
    reti
org 1bh ; "заглушка" для Tmr1_ovf
    nop
    reti
org 23h ; "заглушка" для UART
    nop
    reti
org 2bh
START:
; Инициализация МК *****
    mov SP, #07h
    ; TODO: CS_EN = 1; все столбцы в 0 (иначе нажатие не опустит строку и INT0);
    ; TODO: режим INT0 (IT0), разрешить EX0 и EA
    jmp $ ; %stop%

; KbScan — сканирование матрицы клавиатуры по столбцам (обработчик INT0).
; Вход: нет. Выход: A — код единственной нажатой клавиши по табл. 3 ТЗ или 0FFh.
; TODO: портит ...
KbScan:
    ; TODO: по очереди 0 в один столбец (запись в ADR_KB), чтение строк (младшие KB_ROWS бит),
    ; TODO: код по рис. 4; две клавиши и больше — 0FFh; в конце все столбцы снова в 0
    mov A, #0FFh
    ret
END
```

Программа 2 — файл «Грошев АВ-код-2»

```
; Грошев А.В., А-06-23, 1, v1
$INCLUDE (vars.inc) ; адреса, биты, константы варианта; в файл для робота mpcode вклеит его текст
DSEG AT 30h
; здесь объявление переменных (если нужны)
CSEG AT 0h
    sjmp START
org 03h ; "заглушка" для INT0
    nop
    reti
org 0bh ; "заглушка" для Tmr0_ovf
    nop
    reti
org 13h ; "заглушка" для INT1
    nop
    reti
org 1bh ; "заглушка" для Tmr1_ovf
    nop
    reti
org 23h ; "заглушка" для UART
    nop
    reti
org 2bh
START:
; Инициализация МК *****
    mov SP, #07h
    ; TODO: CS_EN = 1; буфер пуст: HEAD = TAIL = BUF_START, F_EMPTY = 1, F_OVF = 0
    mov A, #55h ; пример отсчёта
    call BufWrite ; %proc%
    jmp $ ; %stop%

; BufWrite — запись отсчёта в кольцевой буфер; если буфер полон — затирается самый старый.
; Вход: A — отсчёт. Выход: F_EMPTY = 0; F_OVF = 1, если отсчёт затёр самый старый.
; TODO: что портит
BufWrite:
    ; TODO: полон (HEAD = TAIL, F_EMPTY = 0) — сдвинуть TAIL и F_OVF = 1; movx по адресу HEAD, HEAD дальше
    (BUF_END → BUF_START)
    ret
END
```

Программа 3 — файл «Грошев АВ-код-3»

```
; Грошев А.В., А-06-23, 1, v1
$INCLUDE (vars.inc)          ; адреса, биты, константы варианта; в файл для робота mpscode вклеит его текст
DSEG AT 30h
; здесь объявление переменных (счётчик тиков и т. п.)
CSEG AT 0h
    sjmp START
org 03h ; "заглушка" для INT0
    nop
    reti
org 0bh ; Timer0 — тик для отсчёта T1
    ljmp T0Isr
org 13h ; "заглушка" для INT1
    nop
    reti
org 1bh ; "заглушка" для Tmr1_ovf
    nop
    reti
org 23h ; "заглушка" для UART
    nop
    reti
org 2bh
START:
; Инициализация МК *****
    mov SP, #07h
    ; TODO: CS_EN = 1; PIN_Y1 = 1 (строб пассивен); TMOD для Timer0; разрешить ET0 и EA
    call Y1Start ; %proc%
    jmp $ ; %stop%

; Y1Start — начало строба Y1 = 0 длительностью T1 на PIN_Y1; конец — в обработчике Timer0.
; Вход, выход: нет.
; TODO: что портит
Y1Start:
    ; TODO: PIN_Y1 = 0, запустить отсчёт Y1_TICKS тиков по TICK_H:TICK_L (в цикле не ждать!)
    ret

; T0Isr — тик Timer0: перезагрузка, счёт тиков, по окончании T1 — PIN_Y1 = 1.
T0Isr:
    ; TODO
    reti
END
```